

EVA Database Schema — v0.3

Research Node

Stage 1 output — Architect Agent

Status: DRAFT — schema is inferred, not confirmed.

Do not run migrations against this until

`core.py` / `models.py` are reviewed.

Legend

✓ CONFIRMED (field/behavior observed directly in `api.py`) · 📋 PLANNED (handoff doc, unverified) · ? OPEN (unknown)

Engine

? Unknown. `main(port, db_path="eva.db")` in `api.py` takes a file path, which is consistent with SQLite, but this is an inference, not a confirmed fact. Handoff doc recommends PostgreSQL + pgvector — **not confirmed as implemented**. Stage 2 must confirm actual engine from `core.py` / `memory/database.py` before writing migrations.

Tables

users

📋 Planned. ✅ Partially confirmed via
`/user/create` : a user has at minimum:

- `id` ✅ (returned in response)
- `name` ✅ (accepted as input, defaults to "anonymous")

missions

📋 Planned table. ✅ Confirmed fields (from
`/mission/create` , `/mission/status` response
shapes):

- `id` ✅
- `status` ✅ (enum, `.value` accessed – string-backed enum; possible values ? unknown, `TaskStatus` is imported but full mission status enum not seen)
- `objective` ✅ (maps from request's `raw_request / objective`)
- `report` ✅ (nullable – "No report yet" handled client-side)
- `user_id` ✅ (required input to `submit_mission`)
- `source_text` ✅ (optional input)

- `sources` ✓ (optional input, list – shape seen in frontend: `{origin, description, text}` per search result mapping)
- `user_confirmed` ✓ (boolean input, used in authorization flow)

tasks

📋 Planned table. ✓ Confirmed fields (from `Task(...)` construction in `api.py`):

- `id` ✓ (`new_id()`)
- `mission_id` ✓ (FK to missions, required)
- `agent` ✓ (`AgentName` enum – members ? unknown beyond usage; presumably includes pixie/nova/sentinel/ghost per handoff doc, unconfirmed)
- `description` ✓
- `auth_level` ✓ (`AuthLevel` enum; confirmed member: `USER_APPROVED`, others ? unknown)
- `status` ✓ (`TaskStatus` enum, `.value` accessed; members ? unknown)

agents

📋 Planned in handoff doc's table list. ? Not directly evidenced in `api.py` – `AgentName` looks like a fixed enum rather than a DB-backed table. **Open question**

for Stage 2: is `agents` a real table, or is agent identity just an enum with no row-level state?

`memory_records` (Ghost)

📋 Planned table, name inferred. ✅ Confirmed fields (from `/memory/store`, `/memory/search`):

- `id` ✅
- `content` ✅ (returned by search; ingest takes `statement` as input – possible field rename between input/storage, ? needs confirmation)
- `classification` ✅ (enum, `.value` accessed)
- `confidence` ✅ (float, `.toFixed(2)` used client-side – confirmed numeric)
- `mission_id` ✅ (required on ingest and used as optional filter on search)
- `source_id` ✅ (optional on ingest)
- `timestamp` 📋 (listed in handoff doc's Ghost description, not directly observed in `api.py` response payload, but likely present)
- conflict tracking: `/memory/store` response includes `conflicts: [c.id for c in result.conflicts]` – ✅ confirms some conflict-detection relationship exists, either self-referential FK on `memory_records` or a separate join, ? structure unknown

`sources`

📋 Planned table (handoff doc). ✅ Indirectly evidenced: `source_id` is accepted as an optional field on `/memory/store`, implying a `sources` table exists to reference. Structure ❓ unknown.

permissions

📋 Planned table (handoff doc). ❓ Not directly evidenced — `AuthLevel` may cover this as an enum rather than a table. Needs `core.py` / `security` module to confirm.

audit_logs

📋 Planned table, name inferred as `audit`. ✅
Confirmed via `/audit/history`:
`core.audit.history(mission_id=...)` returns `events`, and the frontend renders each event with:

- `timestamp` ✅
- `agent` ✅
- `action` ✅
- `result` ✅

Open Questions Blocking Finalization

1. Confirm actual DB engine (SQLite vs. Postgres) from `core.py`.
2. Confirm full enum membership for `AgentName`, `AuthLevel`, `TaskStatus`, and mission status.

3. Confirm whether `agents` and `permissions` are real tables or enums-only.
4. Confirm relationship between `ingest(statement=...)` input and `content` field in stored/returned records – same field, different name, or a transform step?
5. Confirm structure of `conflicts` – self-join on `memory_records`, or separate table?

Stage 2 should treat every  field above as locked. Everything marked  or  needs a direct read of `core.py` and `models.py` before any migration is written.